

TFM Churn E-commerce — Bitácora de avance (Parte 1)

Documento de seguimiento del proyecto `tfm-churn-ecommerce`. Registra decisiones, estructura y estado de cada paso del plan (Documento Maestro).

1. Contexto del proyecto

- **Nombre del proyecto:** `tfm-churn-ecommerce`
- **Objetivo:** Predicción de churn (abandono de clientes) en e-commerce.
- **Dataset:** Online Retail II (UCI, ID 502).
- **Gestión de dependencias:** `uv` + `pyproject.toml`.
- **Entorno:** Python 3.12.
- **MLOps:** MLflow se incorporará después de establecer el modelo baseline.
- **Motivo de reinicio:** la estructura previa (generada con ayuda de Copilot) tenía carpetas duplicadas y confusas (`labels / labeling`, `split / splitting`, `.egg-info`, etc.). Se decidió reiniciar desde cero con estructura y código validados manualmente.

2. Estructura del proyecto (validada)

```
tfm-churn-ecommerce/
├── data/
│   ├── raw/           # dataset original (online_retail_II.xlsx)
│   ├── interim/       # cachés intermedias (parquet)
│   └── processed/     # datasets finales (features + labels)
├── notebooks/        # exploración y auditoría
│   └── 01_data_audit.ipynb
├── src/
│   ├── config/        # rutas y parámetros globales (settings.py)
│   ├── data/          # loader.py, cleaning.py
│   ├── labeling/      # definición de churn (pendiente)
│   ├── features/      # RFM y comportamiento (pendiente)
│   ├── split/         # split temporal (pendiente)
│   ├── models/        # entrenamiento (pendiente)
│   ├── evaluation/    # métricas (pendiente)
│   ├── explainability/ # SHAP (pendiente)
│   └── api/           # FastAPI (pendiente)
├── tests/             # pytest
│   └── test_cleaning.py
├── scripts/           # utilidades
│   ├── check_structure.py
│   └── verify_setup.py
├── models/            # artefactos entrenados
├── reports/           # gráficos/resultados
├── pyproject.toml
├── .gitignore
└── README.md
```

Verificada mediante `scripts/check_structure.py` → estructura completa, sin elementos faltantes.

3. Paso 0 — Entorno y verificación

Instalación de uv

- **Problema inicial:** `uv` no era reconocido en PowerShell tras la instalación.

- **Solución:** reinstalación con el script oficial (`irm https://astral.sh/uv/install.ps1 | iex`) y verificación de PATH en la sesión activa.
- **Resultado:** `uv 0.12.13` funcionando correctamente.

pyproject.toml

Dependencias principales configuradas:

```
dependencies = [
    "pandas>=2.2",
    "numpy>=1.26",
    "scikit-learn>=1.5",
    "xgboost>=2.1",
    "matplotlib>=3.9",
    "jupyter>=1.0",
    "shap>=0.46",
    "fastapi>=0.115",
    "pydantic>=2.9",
    "uvicorn[standard]>=0.30",
    "pytest>=8.3",
    "openpyxl>=3.1",
    "pyarrow>=17.0",
    "ucimlrepo>=0.0.7",
]
```

Con secciones opcionales para `mlops` (MLflow, diferido) y `dev` (ruff, black, mypy).

También se configuró pytest para reconocer `src/` como paquete:

```
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
```

Incidencias resueltas

Problema	Causa	Solución
<code>shap</code> no se importaba tras <code>uv add shap</code>	Windows bloqueó un <code>.pyd</code> de scikit-learn (Control de aplicaciones / Windows Defender)	Desbloqueo de archivos con <code>Unblock-File</code>
<code>ImportError: cannot import name 'load_raw_data' from 'src'</code>	Desalineación entre <code>__init__.py</code> y el nombre real de la función en <code>loader.py</code>	Reescritura completa de <code>loader.py</code>
<code>ModuleNotFoundError: No module named 'src'</code> al ejecutar el script directo	Python no añade la raíz del proyecto al <code>sys.path</code> al correr un archivo suelto	Ejecutar como módulo: <code>uv run python -m src.data.loader</code>
Mismo error en pytest	pytest tampoco añadía la raíz al path	<code>pythonpath = ["."]</code> en <code>pyproject.toml</code>

scripts/verify_setup.py

Verifica versión de Python, paquetes instalados y presencia del dataset. Resultado final: todo correcto (Python 3.12.14, todos los paquetes, dataset de 43.5 MB presente).

4. Carga de datos — `src/data/loader.py`

Función principal: `load_online_retail(file_path, use_cache=True)`.

- Lee las dos hojas del Excel (Year 2009-2010, Year 2010-2011) y las concatena.
- Valida que existan las columnas esperadas.
- Cachea el resultado combinado en `data/interim/online_retail_ii_raw.parquet` para evitar releer el Excel (~2-3 min) en cada ejecución.

Incidencia: columnas de tipo mixto

Al guardar en Parquet, pyarrow rechazó las columnas `Invoice`, `StockCode` y `Description` por mezclar números y strings en la misma columna (ej. `'C489449'` en cancelaciones, descripciones que son solo un número).

Solución aplicada:

```
for col in ("Invoice", "StockCode", "Description"):
    df_sheet[col] = df_sheet[col].apply(
        lambda x: x if pd.isna(x) else str(x)
    )
```

Se castean a string preservando los NaN reales (evitando convertirlos en el string `"nan"`).

Resultado final de carga

- 1.067.371 filas, 9 columnas (incluye `source_sheet`).
- Nulos: `Customer ID` → 243.007; `Description` → 4.382.
- Ejecución validada con: `uv run python -m src.data.loader`.

5. Auditoría de datos — `notebooks/01_data_audit.ipynb`

Notebook reconstruido desde cero usando `load_online_retail()` como única fuente de carga. Secciones:

1. Carga de datos.
2. Revisión de nulos.
3. Análisis de cancelaciones (`Invoice` con prefijo `C`).
4. Efecto de filtrar por `Customer ID` no nulo.
5. Distribución de `Quantity/Price` en transacciones válidas + detección de `TEST001`.
6. Perfil de clientes y umbral p99 para mayoristas.
7. Resumen de reglas de limpieza validadas.

Hallazgos clave

- **Quantity negativa sin prefijo C:** 3.457 filas — corresponden a ajustes internos (check, damages, given away, etc.), no a ventas reales. El 100% tiene `Customer ID` nulo.
- Filtrar por `Customer ID` no nulo elimina automáticamente todo este ruido: tras el filtro, 0 filas con quantity negativa sin cancelación.
- Filas válidas (`Customer ID` no nulo, sin cancelación): 805.620 (antes de excluir `TEST001`).
- **TEST001:** productos de prueba del sistema — deben excluirse (ej. filas con `Price = 0` y descripción *"This is a test product."*).
- **Perfil de clientes (p99):**
 - Umbral Quantity (p99): 17.506
 - Umbral Spend (p99): 29.704,60
 - Clientes marcados como mayoristas: 70 (1,19 %) del total.
 - Cliente destacado: `14646.0` — mayor volumen, candidato claro a mayorista puro.
- Clientes con más de 1 país registrado: 13 (posible ruido de dirección de envío vs. país de residencia, a revisar en labeling/features).

Reglas de limpieza consolidadas

1. Filtrar `Customer ID` no nulo → elimina ruido de quantity negativa sin cancelación.
 2. Excluir `StockCode == 'TEST001'` → ruido de sistema.
 3. Tratar `Invoice` con prefijo `'C'` como cancelación (no como venta).
 4. Marcar clientes con `total_quantity` o `total_spend` por encima del percentil 99 como `is_wholesale`.
-

6. Código reutilizable — `src/data/cleaning.py`

Las 4 reglas se implementaron como funciones puras, para no repetirlas manualmente en cada notebook:

- `add_cancellation_flag(df)` — marca `is_cancellation`.
- `filter_valid_customers(df)` — elimina `Customer ID` nulo.
- `remove_system_noise(df)` — elimina `TEST001`.
- `get_clean_sales(df)` — pipeline completo de limpieza (`Customer ID` válido + sin cancelaciones + sin ruido de sistema).
- `build_customer_profile(df_valid, wholesale_percentile=0.99)` — perfil por cliente + flag `is_wholesale`.

Uso estándar en cualquier notebook/script:

```
from src.data.loader import load_online_retail
from src.data.cleaning import get_clean_sales, build_customer_profile

df = load_online_retail()
df_valid = get_clean_sales(df)
profile = build_customer_profile(df_valid)
```

7. Tests — `tests/test_cleaning.py`

Cobertura básica con pytest, usando un DataFrame de ejemplo controlado:

- `test_add_cancellation_flag` — valida detección del prefijo `c`.
- `test_filter_valid_customers` — valida eliminación de `Customer ID` nulo.
- `test_remove_system_noise` — valida exclusión de `TEST001`.
- `test_get_clean_sales_excludes_cancellations_noise_and_nulls` — valida el pipeline completo combinado.
- `test_build_customer_profile_columns` — valida columnas y cardinalidad del perfil de cliente.

Resultado: todos los tests pasan (`uv run pytest tests/test_cleaning.py -v`).

8. Estado actual y próximos pasos

✓ Completado

- ☒ Reinicio de estructura de proyecto (limpia, sin duplicados).
- ☒ Entorno `uv` + `pyproject.toml` verificado (`verify_setup.py`).
- ☒ `src/data/loader.py` con caché Parquet, corriendo sin errores.
- ☒ `notebooks/01_data_audit.ipynb` ejecutado de punta a punta.
- ☒ `src/data/cleaning.py` con reglas de limpieza reutilizables.
- ☒ `tests/test_cleaning.py` con cobertura básica, todos los tests en verde.

» Próximos pasos

- ☐ `src/labeling/` — definir la variable objetivo de churn usando los horizontes ya configurados en `settings.py` (`CHURN_HORIZONS` = [60, 90, 120] días).
- ☐ Decidir tratamiento de clientes mayoristas (excluir del modelo, modelar aparte, o incluir como feature `is_wholesale`).
- ☐ `src/features/` — construcción de features RFM y de comportamiento.
- ☐ `src/split/` — split temporal (train/test respetando la cronología).
- ☐ `src/models/` — entrenamiento del modelo baseline.
- ☐ `src/evaluation/` — métricas de evaluación.
- ☐ `src/explainability/` — SHAP.

- ☐ `src/api/` — servicio FastAPI de inferencia.
- ☐ Incorporar MLflow (una vez exista el baseline).

Última actualización: registro generado tras cerrar el ciclo de carga, limpieza y testing de datos (Paso 0 + auditoría inicial).